

Como instalar **Python e VS Code** no Windows

Guia completo para iniciantes em 2026



Autor: Vagner Oliveira Pimentel Pereira

Site oficial: <https://sites.google.com/view/profvagneroliveira>

SUMÁRIO

Introdução

Capítulo 1: Baixando e Instalando o Python

Capítulo 2: Instalando o VS Code

Capítulo 3: Configurando o VS Code para Python

Capítulo 4: Instalando o Pandas

Capítulo 5: Seu Primeiro Código com Pandas

Código-Fonte Completo

Lista de Exercícios

Resolução das Lista de Exercícios

Referências Bibliográficas

INTRODUÇÃO

Se você está começando sua jornada na programação com **Python** e deseja configurar um ambiente profissional no **Windows**, este guia foi feito para você.

Em 2026, as ferramentas evoluíram para serem mais intuitivas, mas a configuração correta ainda é o passo mais importante para evitar dores de cabeça no futuro.

Neste e-book, você aprenderá desde o download básico até a execução do seu primeiro script de análise de dados utilizando a biblioteca **Pandas**.

CAPÍTULO 1: BAIXANDO E INSTALANDO O PYTHON

O Python é a base de tudo. Para instalá-lo corretamente no Windows, siga estes passos:

- 1 Acesse o site oficial: python.org/downloads.
- 2 O site detectará automaticamente seu sistema operacional. Clique no botão

Download Python 3.x.

- 3 **Importante:** Ao abrir o instalador, você verá uma tela como a abaixo.

Marque obrigatoriamente a caixa "Add python.exe to PATH". Isso permite que você use o Python em qualquer lugar do seu computador.



- 4 Clique em **Install Now** e aguarde a conclusão.
- 5 Para testar, abra o Prompt de Comando (CMD) e digite: `python --version`.

CAPÍTULO 2: INSTALANDO O VS CODE

O **Visual Studio Code (VS Code)** é o editor de código mais popular do mundo. Ele é leve, poderoso e altamente personalizável.

- 6 Baixe o instalador em: code.visualstudio.com.
- 7 Execute o arquivo baixado. Durante a instalação, aceite o contrato e clique em avançar.
- 8 Na tela de "Tarefas Adicionais", certifique-se de que as opções **"Add to PATH"** e **"Open with Code"** estejam marcadas.

[Image failed to load: Interface Moderna do VS Code 2026]

- 9 Após a instalação, abra o VS Code.

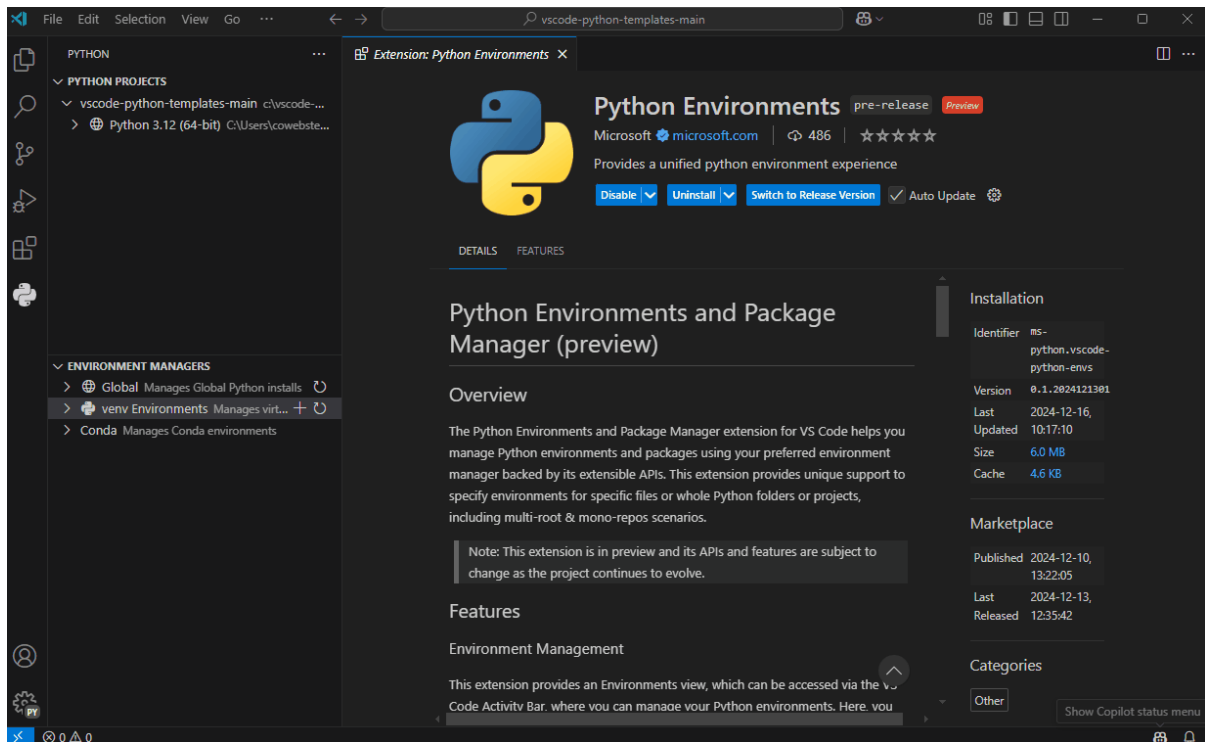
CAPÍTULO 3: CONFIGURANDO O VS CODE PARA PYTHON

O VS Code por si só não "entende" Python profundamente até que você instale a extensão oficial da Microsoft.

10 No VS Code, clique no ícone de **Extensões** na barra lateral esquerda (ou use Ctrl + Shift + X).

11 Pesquise por "**Python**" e selecione a versão desenvolvida pela **Microsoft**.

12 Clique em **Install**.



Esta extensão adiciona suporte para IntelliSense (autocompletar), depuração e navegação de código.

CAPÍTULO 4: INSTALANDO O PANDAS

O **Pandas** é a biblioteca essencial para manipulação de dados em Python. Vamos instalá-la usando o terminal integrado do VS Code.

13 No VS Code, abra um novo terminal em Terminal > New Terminal.

14 Digite o comando abaixo e pressione Enter:

```
pip install pandas
```

15 Aguarde o download e a instalação. Para verificar se deu certo, digite pip show pandas.

CAPÍTULO 5: SEU PRIMEIRO CÓDIGO COM PANDAS

Agora é importante verificar se tudo está funcionando.

16 Crie um novo arquivo chamado primeiro_teste.py.

17 Cole o seguinte código:

```
import pandas as pd

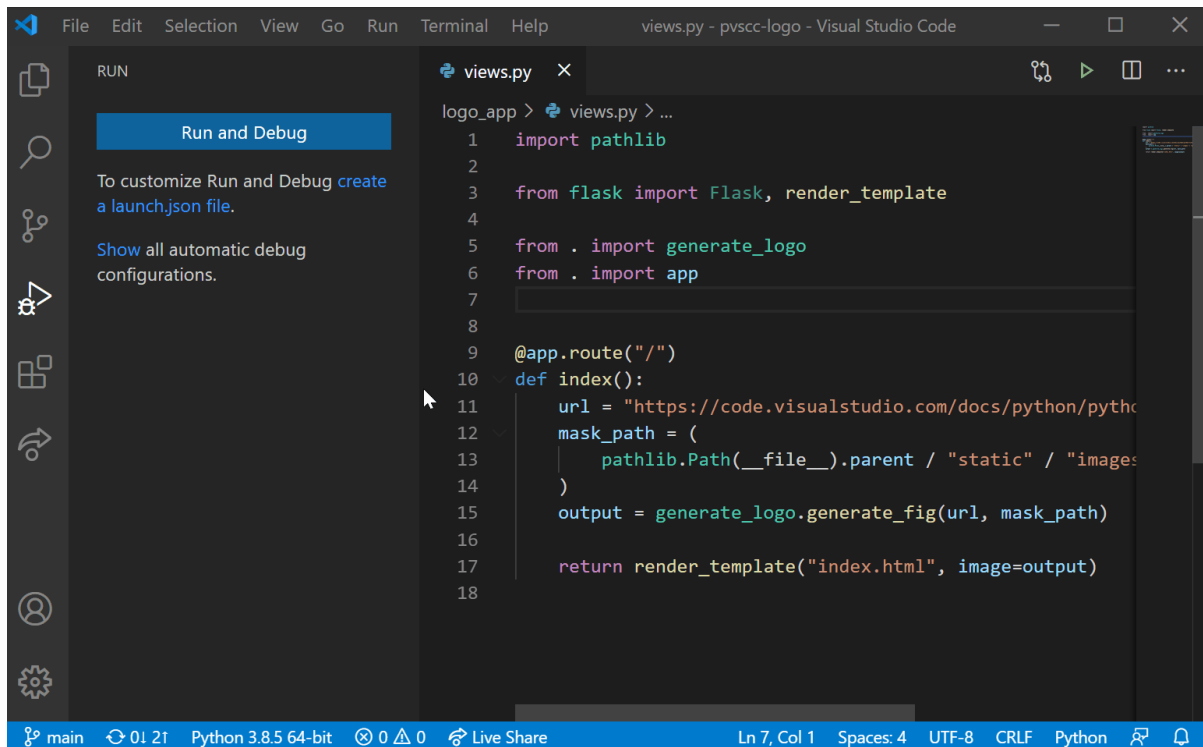
# Criando um conjunto de dados simples
dados = {
    'Nome': ['Vicente', 'Carlos', 'Ana'],
    'Idade': [10, 28, 35],
    'Cidade': ['Salvador', 'São Paulo', 'Rio de Janeiro']
}

# Convertendo para um DataFrame do Pandas
df = pd.DataFrame(dados)

print("Tabela gerada com Pandas:")

print(df)
```

18 Clique no botão de **Play** no canto superior direito ou use o comando python primeiro_teste.py no terminal.



CÓDIGO - FONTE COMPLETO

Abaixo está o código utilizado no exemplo final para referência rápida:

```
import pandas as pd

def main():
    # Estrutura de dados inicial
    dados = {
        'Nome': ['Vicente', 'Carlos', 'Ana'],
        'Idade': [10, 28, 35],
        'Cidade': ['Salvador', 'São Paulo', 'Rio de Janeiro']
    }

    # Criação do DataFrame
    df = pd.DataFrame(dados)

    # Exibição dos resultados
    print("-" * 30)
    print("SISTEMA DE TESTE PYTHON 2026")
    print("-" * 30)
    print(df)
    print("-" * 30)
    print("Ambiente configurado com sucesso!")

if __name__ == "__main__":

    main()
```

LISTA DE EXERCÍCIOS DE PYTHON

Esta lista de exercícios foi elaborada para acompanhar o desenvolvimento de estudantes de Python, abrangendo desde conceitos fundamentais até tópicos mais complexos como manipulação de dados com Pandas e Programação Orientada a Objetos (POO).

Recomenda-se que os exercícios sejam resolvidos em ordem para um aprendizado progressivo.

Nível Básico:

Estes exercícios focam nos conceitos iniciais da linguagem, como variáveis, tipos de dados, operadores, estruturas condicionais e laços de repetição.

Exercício 1: Olá Mundo Personalizado

Crie um programa que solicite o nome do usuário e, em seguida, exiba uma mensagem de boas-vindas personalizada, como "Olá, [Nome do Usuário]! Bem-vindo(a) ao mundo Python!".

Exercício 2: Calculadora Simples

Desenvolva uma calculadora que peça dois números ao usuário e, em seguida, exiba a soma, subtração, multiplicação e divisão entre eles.

Exercício 3: Verificador de Par ou Ímpar

Escreva um programa que receba um número inteiro e determine se ele é par ou ímpar.

Exercício 4: Tabuada

Crie um programa que solicite um número inteiro e exiba a tabuada desse número de 1 a 10.

Exercício 5: Média de Notas

Peça ao usuário para inserir três notas e calcule a média. Informe se o aluno foi aprovado (média ≥ 7) ou reprovado.

Nível Intermediário: Estruturas de Dados e Funções

Nesta seção, os exercícios exploram listas, tuplas, dicionários e a criação e utilização de funções.

Exercício 6: Lista de Compras

Crie um programa que permita ao usuário adicionar, remover e visualizar itens de uma lista de compras. Utilize uma lista para armazenar os itens.

Exercício 7: Contador de Vogais

Escreva uma função que receba uma string como parâmetro e retorne o número de vogais (a, e, i, o, u) presentes nela.

Exercício 8: Dicionário de Contatos

Crie um programa que gerencie uma lista de contatos. Cada contato deve ter nome, telefone e e-mail, armazenados em um dicionário. Permita adicionar, buscar e listar contatos.

Exercício 9: Fatorial de um Número

Desenvolva uma função recursiva para calcular o fatorial de um número inteiro positivo.

Exercício 10: Conversor de Temperatura

Crie funções para converter temperaturas de Celsius para Fahrenheit e vice-versa. O programa deve perguntar ao usuário qual conversão deseja fazer e o valor da temperatura.

Nível Avançado: Manipulação de Arquivos e Módulos

Estes exercícios abordam a leitura e escrita de arquivos, tratamento de exceções e a utilização de módulos externos.

Exercício 11: Leitor de Arquivo de Texto

Crie um programa que leia um arquivo de texto (.txt) e exiba seu conteúdo na tela. O programa deve tratar o erro caso o arquivo não seja encontrado.

Exercício 12: Gerador de Senhas

Desenvolva um gerador de senhas aleatórias. A senha deve ter um comprimento definido pelo usuário e pode incluir letras maiúsculas, minúsculas, números e caracteres especiais. Utilize o módulo random.

Exercício 13: Registro de Logs

Escreva um programa que registre mensagens de log em um arquivo. As mensagens devem incluir um timestamp e o tipo de log (INFO, WARNING, ERROR). Utilize o módulo datetime.

Exercício 14: Validador de CPF

Crie uma função que valide um número de CPF (apenas a lógica de validação, sem interface). Pesquise o algoritmo de validação de CPF.

Exercício 15: Análise de Dados Simples (CSV)

Utilize o módulo csv para ler um arquivo CSV contendo dados de vendas (ex: produto, quantidade, preço). Calcule o total de vendas e o produto mais vendido.

Nível Desafio: Pandas e Programação Orientada a Objetos (POO)

Estes exercícios são para quem já tem uma base sólida e deseja explorar bibliotecas poderosas como Pandas e os princípios da POO.

Exercício 16: Análise de Dados com Pandas

Dado um arquivo CSV (ex: dados_clientes.csv com colunas Nome, Idade, Cidade, Renda):

1. Carregue os dados em um DataFrame do Pandas.
2. Calcule a média de idade e renda dos clientes.
3. Encontre a cidade com o maior número de clientes.
4. Filtre os clientes com renda acima de um valor específico.

Exercício 17: Sistema Bancário (POO)

Crie um sistema bancário simples utilizando Programação Orientada a Objetos. Crie classes como ContaBancaria, Cliente e Banco.

A classe ContaBancaria deve ter atributos como numero_conta, saldo e titular. Métodos para depositar, sacar e verificar_saldo.

A classe Cliente deve ter nome, cpf e uma lista de ContasBancarias.

A classe Banco deve gerenciar uma lista de Clientes e permitir adicionar novos clientes e buscar contas.

Exercício 18: Visualização de Dados com Matplotlib (Opcional)

Utilizando os dados do Exercício 16 (ou outros dados de sua escolha), crie um gráfico de barras da distribuição de clientes por cidade ou um histograma da distribuição de idades, usando a biblioteca matplotlib.

Exercício 19: Web Scraper Simples

Desenvolva um pequena web scraper usando a biblioteca requests e BeautifulSoup para extrair títulos de notícias de um site de notícias (ex: G1, BBC News). Salve os títulos em um arquivo de texto.

Exercício 20: Jogo da Forca (POO)

Implemente o jogo da forca utilizando conceitos de Programação Orientada a Objetos. Crie classes para Jogo, Palavra e Jogador. O jogo deve permitir ao jogador adivinhar letras, exibir o estado atual da palavra e controlar o número de tentativas.

Resolução das Lista de Exercícios

Lembre-se que em programação, muitas vezes, existem múltiplas formas de resolver um problema. Encoraje os alunos a explorar diferentes abordagens e a entender o raciocínio por trás de cada solução.

Nível Básico: Fundamentos de Python

Exercício 1: Olá Mundo Personalizado

```
nome = input("Digite seu nome: ")

print(f"Olá, {nome}! Bem-vindo(a) ao mundo Python!")
```

Exercício 2: Calculadora Simples

```
num1 = float(input("Digite o primeiro número: "))

num2 = float(input("Digite o segundo número: "))

soma = num1 + num2
subtracao = num1 - num2
multiplicacao = num1 * num2

print(f"Soma: {soma}")
print(f"Subtração: {subtracao}")
print(f"Multiplicação: {multiplicacao}")

# Tratamento para divisão por zero
if num2 != 0:
    divisao = num1 / num2
    print(f"Divisão: {divisao}")
else:
    print("Não é possível dividir por zero.")
```

Exercício 3: Verificador de Par ou Ímpar

```
numero = int(input("Digite um número inteiro: "))

if numero % 2 == 0:
    print(f"O número {numero} é PAR.")
```



```
else:
```

```
    print(f"O número {numero} é ÍMPAR.")
```

Exercício 4: Tabuada

```
numero = int(input("Digite um número para ver a tabuada: "))
```

```
print(f"Tabuada do {numero}:")  
for i in range(1, 11):
```

```
    print(f"{numero} x {i} = {numero * i}")
```

Exercício 5: Média de Notas

```
nota1 = float(input("Digite a primeira nota: "))
```

```
nota2 = float(input("Digite a segunda nota: "))  
nota3 = float(input("Digite a terceira nota: "))
```

```
media = (nota1 + nota2 + nota3) / 3
```

```
print(f"Média: {media:.2f}")
```

```
if media >= 7:  
    print("Aluno APROVADO!")  
else:
```

```
    print("Aluno REPROVADO.")
```

Exercício 6: Lista de Compras

```
lista_compras = []
```

```
while True:  
    print("\n--- Menu Lista de Compras ---")  
    print("1. Adicionar item")  
    print("2. Remover item")  
    print("3. Visualizar lista")  
    print("4. Sair")
```

```
    opcao = input("Escolha uma opção: ")
```

```

if opcao == '1':
    item = input("Digite o item a adicionar: ")
    lista_compras.append(item)
    print(f'{item}' adicionado à lista.")
elif opcao == '2':
    item = input("Digite o item a remover: ")
    if item in lista_compras:
        lista_compras.remove(item)
        print(f'{item}' removido da lista.")
    else:
        print(f'{item}' não encontrado na lista.")
elif opcao == '3':
    if lista_compras:
        print("\n--- Sua Lista de Compras ---")
        for i, item in enumerate(lista_compras):
            print(f"{i+1}. {item}")
    else:
        print("Sua lista de compras está vazia.")
elif opcao == '4':
    print("Saindo do programa.")
    break
else:

```

```

print("Opção inválida. Tente novamente.")

```

Exercício 7: Contador de Vogais

```

def contar_vogais(texto):

```

```

    vogais = "aeiouAEIOU"
    contador = 0
    for letra in texto:
        if letra in vogais:
            contador += 1
    return contador

```

```

frase = input("Digite uma frase: ")
num_vogais = contar_vogais(frase)

```

```

print(f"A frase possui {num_vogais} vogais.")

```

Exercício 8: Dicionário de Contatos

```

contatos = {}

```

```

def adicionar_contato(nome, telefone, email):
    contatos[nome] = {"telefone": telefone, "email": email}

```

```

print(f"Contato {nome} adicionado.")

def buscar_contato(nome):
    if nome in contatos:
        print(f"Nome: {nome}")
        print(f"Telefone: {contatos[nome]['telefone']}")
        print(f"Email: {contatos[nome]['email']}")
    else:
        print(f"Contato {nome} não encontrado.")

def listar_contatos():
    if contatos:
        print("\n--- Lista de Contatos ---")
        for nome, dados in contatos.items():
            print(f"Nome: {nome}, Telefone: {dados['telefone']}, Email: {dados['email']}")
    else:
        print("Nenhum contato cadastrado.")

# Exemplo de uso
adicionar_contato("Ana", "11987654321", "ana@email.com")
adicionar_contato("Bruno", "21912345678", "bruno@email.com")
listar_contatos()
buscar_contato("Ana")

buscar_contato("Carlos")

```

Exercício 9: Fatorial de um Número

```

def fatorial(n):

    if n == 0 or n == 1:
        return 1
    else:
        return n * fatorial(n-1)

numero = int(input("Digite um número inteiro positivo para calcular o fatorial: "))
if numero < 0:
    print("Fatorial não é definido para números negativos.")
else:
    print(f"O fatorial de {numero} é {fatorial(numero)}.")

```

Exercício 10: Conversor de Temperatura

```

def celsius_para_fahrenheit(celsius):

    return (celsius * 9/5) + 32

```

```
def fahrenheit_para_celsius(fahrenheit):
    return (fahrenheit - 32) * 5/9

print("\n--- Conversor de Temperatura ---")
print("1. Celsius para Fahrenheit")
print("2. Fahrenheit para Celsius")
opcao = input("Escolha a conversão (1 ou 2): ")

if opcao == '1':
    temp_c = float(input("Digite a temperatura em Celsius: "))
    temp_f = celsius_para_fahrenheit(temp_c)
    print(f"{temp_c}°C é igual a {temp_f:.2f}°F.")
elif opcao == '2':
    temp_f = float(input("Digite a temperatura em Fahrenheit: "))
    temp_c = fahrenheit_para_celsius(temp_f)
    print(f"{temp_f}°F é igual a {temp_c:.2f}°C.")
else:
    print("Opção inválida.")
```

Exercício 11: Leitor de Arquivo de Texto

```
def ler_arquivo(nome_arquivo):

    try:
        with open(nome_arquivo, 'r', encoding='utf-8') as arquivo:
            conteudo = arquivo.read()
            print("\n--- Conteúdo do Arquivo ---")
            print(conteudo)
    except FileNotFoundError:
        print(f"Erro: O arquivo '{nome_arquivo}' não foi encontrado.")
    except Exception as e:
        print(f"Ocorreu um erro ao ler o arquivo: {e}")

# Crie um arquivo 'exemplo.txt' para testar
# Exemplo de uso:
# with open('exemplo.txt', 'w') as f:
#     f.write('Esta é uma linha de teste.\n')
#     f.write('Esta é a segunda linha.')

nome_do_arquivo = input("Digite o nome do arquivo de texto (ex: exemplo.txt): ")

ler_arquivo(nome_do_arquivo)
```

Exercício 12: Gerador de Senhas

```
import random
```

```

import string

def gerar_senha(tamanho, incluir_maiusculas=True, incluir_minusculas=True, incluir_numeros=True,
incluir_especiais=True):
    caracteres = ""
    if incluir_maiusculas: caracteres += string.ascii_uppercase
    if incluir_minusculas: caracteres += string.ascii_lowercase
    if incluir_numeros: caracteres += string.digits
    if incluir_especiais: caracteres += string.punctuation

    if not caracteres:
        return "Erro: Nenhuma opção de caractere selecionada."

    senha = ''.join(random.choice(caracteres) for i in range(tamanho))
    return senha

# Exemplo de uso
tam = int(input("Tamanho da senha: "))
mai = input("Incluir maiúsculas? (s/n): ").lower() == 's'
min = input("Incluir minúsculas? (s/n): ").lower() == 's'
num = input("Incluir números? (s/n): ").lower() == 's'
esp = input("Incluir especiais? (s/n): ").lower() == 's'

senha_gerada = gerar_senha(tam, mai, min, num, esp)

print(f"Senha gerada: {senha_gerada}")

```

Exercício 13: Registro de Logs

```

import datetime

def registrar_log(mensagem, tipo='INFO', nome_arquivo='app.log'):
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    linha_log = f"[{timestamp}] [{tipo}] {mensagem}\n"
    with open(nome_arquivo, 'a', encoding='utf-8') as f:
        f.write(linha_log)

# Exemplo de uso
registrar_log("Aplicação iniciada.", "INFO")
registrar_log("Variável 'x' não definida.", "WARNING")
registrar_log("Falha na conexão com o banco de dados.", "ERROR")

print("Logs registrados em 'app.log'.")

```

Exercício 14: Validador de CPF

```
def validar_cpf(cpf):

    cpf = ''.join(filter(str.isdigit, cpf)) # Remove caracteres não numéricos

    if len(cpf) != 11 or len(set(cpf)) == 1: # Verifica se tem 11 dígitos e não são todos iguais
        return False

    # Validação do primeiro dígito verificador
    soma = 0
    for i in range(9):
        soma += int(cpf[i]) * (10 - i)
    resto = soma % 11
    digito1 = 0 if resto < 2 else 11 - resto

    if digito1 != int(cpf[9]):
        return False

    # Validação do segundo dígito verificador
    soma = 0
    for i in range(10):
        soma += int(cpf[i]) * (11 - i)
    resto = soma % 11
    digito2 = 0 if resto < 2 else 11 - resto

    if digito2 != int(cpf[10]):
        return False

    return True

# Exemplo de uso
cpf_teste = input("Digite um CPF para validar: ")
if validar_cpf(cpf_teste):
    print(f"O CPF {cpf_teste} é VÁLIDO.")
else:
    print(f"O CPF {cpf_teste} é INVÁLIDO.")
```

Exercício 15: Análise de Dados Simples (CSV)

```
import csv

def analisar_vendas(nome_arquivo):
    total_vendas = 0
    produtos_vendidos = {}
```

```

try:
    with open(nome_arquivo, 'r', encoding='utf-8') as arquivo_csv:
        leitor = csv.DictReader(arquivo_csv)
        for linha in leitor:
            produto = linha['produto']
            quantidade = int(linha['quantidade'])
            preco = float(linha['preco'])

            total_vendas += quantidade * preco
            produtos_vendidos[produto] = produtos_vendidos.get(produto, 0) + quantidade

        if produtos_vendidos:
            produto_mais_vendido = max(produtos_vendidos, key=produtos_vendidos.get)
            print(f"\n--- Análise de Vendas ---")
            print(f"Total de vendas: R$ {total_vendas:.2f}")
            print(f"Produto mais vendido: {produto_mais_vendido}
({produtos_vendidos[produto_mais_vendido]} unidades)")
        else:
            print("Nenhum dado de vendas encontrado.")

    except FileNotFoundError:
        print(f"Erro: O arquivo '{nome_arquivo}' não foi encontrado.")
    except KeyError as e:
        print(f"Erro: Coluna '{e}' não encontrada no arquivo CSV. Verifique o cabeçalho.")
    except ValueError as e:
        print(f"Erro de conversão de dados: {e}. Verifique os valores numéricos no CSV.")
    except Exception as e:
        print(f"Ocorreu um erro inesperado: {e}")

# Crie um arquivo 'vendas.csv' para testar:
# produto,quantidade,preco
# camiseta,10,25.50
# calca,5,80.00
# tenis,3,150.00
# camiseta,2,25.50

analisar_vendas('vendas.csv')

```

Exercício 16: Análise de Dados com Pandas

```

import pandas as pd

# Crie um arquivo 'dados_clientes.csv' para testar:
# Nome,Idade,Cidade,Renda
# Ana,28,São Paulo,5000.00
# Bruno,35,Rio de Janeiro,7500.00
# Carlos,22,São Paulo,3000.00

```

```

# Daniela,40,Belo Horizonte,9000.00
# Eduardo,30,Rio de Janeiro,6000.00

try:
    df = pd.read_csv('dados_clientes.csv')

    print("\n--- Análise de Dados de Clientes ---")
    print("DataFrame Original:")
    print(df)

    # 2. Calcular a média de idade e renda
    media_idade = df['Idade'].mean()
    media_renda = df['Renda'].mean()
    print(f"\nMédia de Idade: {media_idade:.2f} anos")
    print(f"Média de Renda: R$ {media_renda:.2f}")

    # 3. Encontrar a cidade com o maior número de clientes
    cidade_mais_clientes = df['Cidade'].value_counts().idxmax()
    print(f"\nCidade com o maior número de clientes: {cidade_mais_clientes}")

    # 4. Filtrar clientes com renda acima de um valor específico
    renda_minima = float(input("\nDigite a renda mínima para filtrar clientes: R$ "))
    clientes_alta_renda = df[df['Renda'] > renda_minima]
    print(f"\nClientes com renda acima de R$ {renda_minima:.2f}:")
    print(clientes_alta_renda)

except FileNotFoundError:
    print("Erro: O arquivo 'dados_clientes.csv' não foi encontrado.")
except KeyError as e:
    print(f"Erro: Coluna '{e}' não encontrada no arquivo CSV. Verifique o cabeçalho.")
except Exception as e:
    print(f"Ocorreu um erro inesperado: {e}")

```

Exercício 17: Sistema Bancário (POO)

```

class ContaBancaria:

    def __init__(self, numero_conta, titular, saldo_inicial=0):
        self.numero_conta = numero_conta
        self.titular = titular
        self.saldo = saldo_inicial

    def depositar(self, valor):
        if valor > 0:
            self.saldo += valor
            print(f"Depósito de R$ {valor:.2f} realizado. Novo saldo: R$ {self.saldo:.2f}")
        else:
            print("Valor de depósito inválido.")

```



```

def sacar(self, valor):
    if valor > 0 and self.saldo >= valor:
        self.saldo -= valor
        print(f"Saque de R$ {valor:.2f} realizado. Novo saldo: R$ {self.saldo:.2f}")
        return True
    elif valor > self.saldo:
        print("Saldo insuficiente.")
        return False
    else:
        print("Valor de saque inválido.")
        return False

def verificar_saldo(self):
    print(f"Saldo atual da conta {self.numero_conta}: R$ {self.saldo:.2f}")
    return self.saldo

class Cliente:
    def __init__(self, nome, cpf):
        self.nome = nome
        self.cpf = cpf
        self.contas = [] # Lista de objetos ContaBancaria

    def adicionar_conta(self, conta):
        self.contas.append(conta)
        print(f"Conta {conta.numero_conta} adicionada ao cliente {self.nome}.")

    def listar_contas(self):
        if self.contas:
            print(f"\nContas de {self.nome}:")
            for conta in self.contas:
                print(f"  Número: {conta.numero_conta}, Saldo: R$ {conta.saldo:.2f}")
        else:
            print(f"O cliente {self.nome} não possui contas.")

class Banco:
    def __init__(self, nome):
        self.nome = nome
        self.clientes = [] # Lista de objetos Cliente

    def adicionar_cliente(self, cliente):
        self.clientes.append(cliente)
        print(f"Cliente {cliente.nome} adicionado ao banco {self.nome}.")

    def buscar_cliente_por_cpf(self, cpf):
        for cliente in self.clientes:
            if cliente.cpf == cpf:
                return cliente
        return None

    def buscar_conta_por_numero(self, numero_conta):

```

```

for cliente in self.clientes:
    for conta in cliente.contas:
        if conta.numero_conta == numero_conta:
            return conta
    return None

# Exemplo de uso do sistema bancário
banco_digital = Banco("Meu Banco Digital")

cliente1 = Cliente("Maria Silva", "123.456.789-00")
conta1_maria = ContaBancaria("001-X", "Maria Silva", 1000)
conta2_maria = ContaBancaria("002-Y", "Maria Silva", 500)
cliente1.adicionar_conta(conta1_maria)
cliente1.adicionar_conta(conta2_maria)
banco_digital.adicionar_cliente(cliente1)

cliente2 = Cliente("João Souza", "987.654.321-00")
conta1_joao = ContaBancaria("003-Z", "João Souza", 2000)
cliente2.adicionar_conta(conta1_joao)
banco_digital.adicionar_cliente(cliente2)

# Operações
cliente1.listar_contas()
conta1_maria.depositar(200)
conta1_maria.sacar(300)
conta1_maria.verificar_saldo()

cliente_encontrado = banco_digital.buscar_cliente_por_cpf("123.456.789-00")
if cliente_encontrado:
    print(f"\nCliente encontrado: {cliente_encontrado.nome}")

conta_encontrada = banco_digital.buscar_conta_por_numero("003-Z")
if conta_encontrada:
    print(f"Conta encontrada: {conta_encontrada.numero_conta} do titular {conta_encontrada.titular}")

```

Exercício 18: Visualização de Dados com Matplotlib (Opcional)

```

import pandas as pd

import matplotlib.pyplot as plt

# Reutilizando o DataFrame do Exercício 16 ou criando um novo
# Certifique-se de ter 'dados_clientes.csv' ou crie um DataFrame aqui
data = {
    'Nome': ['Ana', 'Bruno', 'Carlos', 'Daniela', 'Eduardo', 'Fernanda', 'Gustavo'],
    'Idade': [28, 35, 22, 40, 30, 25, 33],
    'Cidade': ['São Paulo', 'Rio de Janeiro', 'São Paulo', 'Belo Horizonte', 'Rio de Janeiro', 'São Paulo', 'Curitiba'],
}

```

```

'Renda': [5000.00, 7500.00, 3000.00, 9000.00, 6000.00, 4500.00, 8000.00]
}
df_vis = pd.DataFrame(data)

# Gráfico de barras: Distribuição de clientes por cidade
plt.figure(figsize=(10, 6))
df_vis['Cidade'].value_counts().plot(kind='bar', color='skyblue')
plt.title('Número de Clientes por Cidade')
plt.xlabel('Cidade')
plt.ylabel('Número de Clientes')
plt.xticks(rotation=45)
plt.grid(axis='y', linestyle='--')
plt.tight_layout()
plt.savefig('clientes_por_cidade.png') # Salva o gráfico como imagem
plt.show() # Exibe o gráfico (pode não funcionar em todos os ambientes sem display)

# Histograma: Distribuição de idades
plt.figure(figsize=(10, 6))
plt.hist(df_vis['Idade'], bins=5, color='lightcoral', edgecolor='black')
plt.title('Distribuição de Idades dos Clientes')
plt.xlabel('Idade')
plt.ylabel('Frequência')
plt.grid(axis='y', linestyle='--')
plt.tight_layout()
plt.savefig('distribuicao_idades.png') # Salva o gráfico como imagem
plt.show() # Exibe o gráfico

print("Gráficos 'clientes_por_cidade.png' e 'distribuicao_idades.png' gerados.")

```

Exercício 19: Web Scraper Simples

```

import requests

from bs4 import BeautifulSoup

def extrair_titulos_noticias(url, nome_arquivo='noticias.txt'):
    try:
        response = requests.get(url)
        response.raise_for_status() # Lança exceção para erros HTTP

        soup = BeautifulSoup(response.text, 'html.parser')

        # Adapte o seletor CSS/HTML de acordo com o site alvo
        # Exemplo para um site genérico que usa <h2> para títulos de notícias
        titulos = soup.find_all('h2') # Pode ser 'h1', 'h3', 'a', 'div', etc., com classes específicas

        with open(nome_arquivo, 'w', encoding='utf-8') as f:
            for titulo in titulos:

```

```

texto_titulo = titulo.get_text(strip=True)
if texto_titulo:
    f.write(texto_titulo + '\n')
print(f"Títulos de notícias extraídos e salvos em '{nome_arquivo}'.")

except requests.exceptions.RequestException as e:
    print(f"Erro de conexão ou HTTP: {e}")
except Exception as e:
    print(f"Ocorreu um erro: {e}")

# Exemplo de uso (substitua pela URL de um site de notícias real)
# Cuidado ao fazer scraping, verifique os termos de serviço do site.
# url_noticias = "https://g1.globo.com/"
# extrair_titulos_noticias(url_noticias)

print("Para testar o Web Scraper, descomente as linhas finais e substitua a URL por um site de notícias real.")

```

Exercício 20: Jogo da Forca (POO)

```
import random

class JogoDaForca:
    def __init__(self, palavras):
        self.palavras = palavras
        self.palavra_secreta = self._escolher_palavra()
        self.letras_corretas = []
        self.letras_erradas = []
        self.tentativas_restantes = 6 # Número de partes do corpo da forca
        self.estado_forca = [
            """ +---+
            |
            |
            |
            |
            =
            """,
            """ +---+
            |
            O |
            |
            |
            =
            """,
            """ +---+
            |
            O |
            |
            |
            =
            """]
```

```

    |
    =
    """
        """ +----+

    | |
    0 |
    /| |
    |
    =
    """
        """ +----+

    | |
    0 |
    /\ |
    |
    =
    """
        """ +----+

    | |
    0 |
    /\ |
    / |
    =
    """
        """ +----+

    | |
    0 |
    /\ |
    /\ |
    =
    """
]

def _escolher_palavra(self):
    return random.choice(self.palavras).upper()

def _exibir_palavra_escondida(self):
    exibicao = ""
    for letra in self.palavra_secreta:
        if letra in self.letras_corretas:
            exibicao += letra + " "
        else:
            exibicao += "_ "
    return exibicao.strip()

def _desenhar_forca(self):
    print(self.estado_forca[6 - self.tentativas_restantes])

def jogar(self):
    print("\n--- Jogo da Forca --- ")
    print("Adivinhe a palavra!")

```

```
while self.tentativas_restantes > 0:
    self._desenhar_forca()
    print(f"Palavra: {self._exibir_palavra_escondida()}")
    print(f"Letras erradas: {' '.join(self.letras_erradas)}")
    print(f"Tentativas restantes: {self.tentativas_restantes}")

    tentativa = input("Digite uma letra: ").upper()

    if len(tentativa) != 1 or not tentativa.isalpha():
        print("Entrada inválida. Digite apenas uma letra.")
        continue

    if tentativa in self.letras_corretas or tentativa in self.letras_erradas:
        print(f"Você já tentou a letra '{tentativa}'.")
        continue

    if tentativa in self.palavra_secreta:
        self.letras_corretas.append(tentativa)
        print(f"Boa! A letra '{tentativa}' está na palavra.")
    else:
        self.letras_erradas.append(tentativa)
        self.tentativas_restantes -= 1
        print(f"Que pena! A letra '{tentativa}' não está na palavra.")

    if "_" not in self._exibir_palavra_escondida():
        print("\nParabéns! Você adivinhou a palavra!")
        print(f"A palavra era: {self.palavra_secreta}")
        self._desenhar_forca()
        break
    else:
        self._desenhar_forca()
        print("\nGAME OVER! Você ficou sem tentativas.")
        print(f"A palavra secreta era: {self.palavra_secreta}")
```

```
# Lista de palavras para o jogo
palavras_forca = ["PYTHON", "PROGRAMACAO", "DESENVOLVIMENTO", "COMPUTADOR",
                  "ALGORITMO", "INTELIGENCIA"]
jogo = JogoDaForca(palavras_forca)
```

```
jogo.jogar()
```

Referências Bibliográficas

Para este guia, foram consultadas fontes oficiais e tutoriais de referência com datas variadas para garantir a precisão:

Fonte	Descrição	Ano	Link
Python Software Foundation	Documentação Oficial do Python 3.14	2026	docs.python.org
Microsoft VS Code	Guia de Configuração Python no Windows	2026	code.visualstudio.com
Microsoft Learn	Introdução ao Python no Windows	2025	learn.microsoft.com
Pandas Documentation	Manual de Referência da Biblioteca Pandas	2024	pandas.pydata.org
DataCamp	Tutorial de Configuração de Ambiente VS Code	2024	datacamp.com
Kinsta Blog	Guia Completo de Instalação do Python	2023	kinsta.com
VS Code Docs	Setup Guide for Windows (Archive)	2021	code.visualstudio.com